

מעבדה בתכנון ספרתי מתקדם

הכנה 4

מגישים:

אלעד אסבג – 318530482

יוסי ברים- 315512848

צוות 7

1.

הלוח שלנו מספק 4 ספרות ב-7 סיגמנטס בעוד שאנו צריכים 6 ספרות בשביל השעות, דקות והשניות.

2.

(א)

יש לנו 2 ספרות כאשר ה-7 סיגמנט מחובר לשניהם ועובד בשיטת הקטודה משותפת וצריך לבחור לפי האנודה איזה לדים יפעלו כאשר ניתן לה 1 לוגי. כדי לשלוט על ספרה אחת בנפרד נפעיל את פין

CAT כאשר הוא יהיה 0 הספרה השמאלית תהיה מוצגת. וכן להפך ב-1 לוגי.

(ב)

כדי לחבר את ה-SSD לבורד שלנו, נצטרך לחבר ל-PMODE שלנו. כאשר יספיק לנו PMODE 1 בעל 8 כניסות. מה שכן נצטרך להפוך את הכניסות כי כל אחד פועל בצורה הפוכה.

(ג)

יש כל מיני שיקולים. אם מדובר בשעון נעדיף להראות את השעות והדקות, ואתה השניות בחיצוני. מישום שהוא פחות חשוב. אבל אם מדברים על טיימר שצריך דיוק כמו במיקרוגל, נרצה להראות את הדקות והשניות.

בכל מקרה אצלינו יהיה יותר נוח שהדקות והשניות יהיו בבור כדי לבדוק את התקינות בזמן סביר. ומשום שהם באותו הלוגיקה של מחזור של 59.

.3
(X

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity BINARY_TO_7SEGMENT is
6  port(
7      B_IN          : in    STD_LOGIC_VECTOR(3 downto 0);
8      seven_segment : out   STD_LOGIC_VECTOR(6 downto 0);
9  );
10
11  end BINARY_TO_7SEGMENT;
12
13  architecture BINARY_TO_7SEGMENT_ARC of BINARY_TO_7SEGMENT is
14
15  begin
16
17  process(B_in)
18  begin
19
20      case B_IN is
21          when "0000" => seven_segment <= "0000001"; --- 0
22          when "0001" => seven_segment <= "1001111"; --- 1
23          when "0010" => seven_segment <= "0010010"; --- 2
24          when "0011" => seven_segment <= "0000110"; --- 3
25          when "0100" => seven_segment <= "1001100"; --- 4
26          when "0101" => seven_segment <= "0100100"; --- 5
27          when "0110" => seven_segment <= "0100000"; --- 6
28          when "0111" => seven_segment <= "0001111"; --- 7
29          when "1000" => seven_segment <= "0000000"; --- 8
30          when "1001" => seven_segment <= "0000100"; --- 9
31          when others => seven_segment <= "XXXXXXX"; --- error
32      end case;
33
34  end process;
35
36  end BINARY_TO_7SEGMENT_ARC;
```

(b

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity BINARY_TO_7SEGMENT_SSD is
6  port(
7      B_IN          : in    STD_LOGIC_VECTOR(3 downto 0);
8      seven_segment : out   STD_LOGIC_VECTOR(6 downto 0);
9  );
10
11  end BINARY_TO_7SEGMENT_SSD;
12
13  architecture BINARY_TO_7SEGMENT_SSD_ARC of BINARY_TO_7SEGMENT_SSD is
14
15  begin
16
17  process(B_in)
18  begin
19
20      case B_IN is
21          when "0000" => seven_segment <= "1111110"; --- 0
22          when "0001" => seven_segment <= "0110000"; --- 1
23          when "0010" => seven_segment <= "1101101"; --- 2
24          when "0011" => seven_segment <= "1111001"; --- 3
25          when "0100" => seven_segment <= "0110011"; --- 4
26          when "0101" => seven_segment <= "1011011"; --- 5
27          when "0110" => seven_segment <= "1011111"; --- 6
28          when "0111" => seven_segment <= "1110000"; --- 7
29          when "1000" => seven_segment <= "1111111"; --- 8
30          when "1001" => seven_segment <= "1111011"; --- 9
31          when others => seven_segment <= "XXXXXXX"; --- error
32      end case;
33
34  end process;
35
36  end BINARY_TO_7SEGMENT_SSD_ARC;
```

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity BIN_TO_BD is
6  port(
7      VALUE_BIN : in  STD_LOGIC_VECTOR(5 downto 0);
8      MS       : out STD_LOGIC_VECTOR(3 downto 0);
9      LS       : out STD_LOGIC_VECTOR(3 downto 0);
10 );
11
12 end BIN_TO_BD;
13
14 architecture BIN_TO_BD_ARC of BIN_TO_BD is
15
16 begin
17
18 process(VALUE_BIN)
19 begin
20
21     case VALUE_BIN is
22         when "000000" => MS <= "x"0" ;LS <= "x"0" ;
23         when "000001" => MS <= "x"0" ;LS <= "x"1" ;
24         when "000010" => MS <= "x"0" ;LS <= "x"2" ;
25         when "000011" => MS <= "x"0" ;LS <= "x"3" ;
26         when "000100" => MS <= "x"0" ;LS <= "x"4" ;
27         when "000101" => MS <= "x"0" ;LS <= "x"5" ;
28         when "000110" => MS <= "x"0" ;LS <= "x"6" ;
29         when "000111" => MS <= "x"0" ;LS <= "x"7" ;
30         when "001000" => MS <= "x"0" ;LS <= "x"8" ;
31         when "001001" => MS <= "x"0" ;LS <= "x"9" ;
32         when "001010" => MS <= "x"1" ;LS <= "x"0" ;
33         when "001011" => MS <= "x"1" ;LS <= "x"1" ;
34         when "001100" => MS <= "x"1" ;LS <= "x"2" ;
35         when "001101" => MS <= "x"1" ;LS <= "x"3" ;
36         when "001110" => MS <= "x"1" ;LS <= "x"4" ;
37         when "001111" => MS <= "x"1" ;LS <= "x"5" ;
38         when "010000" => MS <= "x"1" ;LS <= "x"6" ;
39         when "010001" => MS <= "x"1" ;LS <= "x"7" ;
40         when "010010" => MS <= "x"1" ;LS <= "x"8" ;
41         when "010011" => MS <= "x"1" ;LS <= "x"9" ;
42         when "010100" => MS <= "x"2" ;LS <= "x"0" ;
43         when "010101" => MS <= "x"2" ;LS <= "x"1" ;
44         when "010110" => MS <= "x"2" ;LS <= "x"2" ;
45         when "010111" => MS <= "x"2" ;LS <= "x"3" ;
46         when "011000" => MS <= "x"2" ;LS <= "x"4" ;
47         when "011001" => MS <= "x"2" ;LS <= "x"5" ;
48         when "011010" => MS <= "x"2" ;LS <= "x"6" ;
49         when "011011" => MS <= "x"2" ;LS <= "x"7" ;
50         when "011100" => MS <= "x"2" ;LS <= "x"8" ;
51         when "011101" => MS <= "x"2" ;LS <= "x"9" ;
52         when "011110" => MS <= "x"3" ;LS <= "x"0" ;
53         when "011111" => MS <= "x"3" ;LS <= "x"1" ;
54         when "100000" => MS <= "x"3" ;LS <= "x"2" ;
55         when "100001" => MS <= "x"3" ;LS <= "x"3" ;
56         when "100010" => MS <= "x"3" ;LS <= "x"4" ;
57         when "100011" => MS <= "x"3" ;LS <= "x"5" ;
58         when "100100" => MS <= "x"3" ;LS <= "x"6" ;
59         when "100101" => MS <= "x"3" ;LS <= "x"7" ;
60         when "100110" => MS <= "x"3" ;LS <= "x"8" ;
61         when "100111" => MS <= "x"3" ;LS <= "x"9" ;
62         when "101000" => MS <= "x"4" ;LS <= "x"0" ;
63         when "101001" => MS <= "x"4" ;LS <= "x"1" ;
64         when "101010" => MS <= "x"4" ;LS <= "x"2" ;
65         when "101011" => MS <= "x"4" ;LS <= "x"3" ;
66         when "101100" => MS <= "x"4" ;LS <= "x"4" ;
67         when "101101" => MS <= "x"4" ;LS <= "x"5" ;
68         when "101110" => MS <= "x"4" ;LS <= "x"6" ;
69         when "101111" => MS <= "x"4" ;LS <= "x"7" ;
70         when "110000" => MS <= "x"4" ;LS <= "x"8" ;
71         when "110001" => MS <= "x"4" ;LS <= "x"9" ;
72         when "110010" => MS <= "x"5" ;LS <= "x"0" ;
73         when "110011" => MS <= "x"5" ;LS <= "x"1" ;
74         when "110100" => MS <= "x"5" ;LS <= "x"2" ;
75         when "110101" => MS <= "x"5" ;LS <= "x"3" ;
76         when "110110" => MS <= "x"5" ;LS <= "x"4" ;
77         when "110111" => MS <= "x"5" ;LS <= "x"5" ;
78         when "111000" => MS <= "x"5" ;LS <= "x"6" ;
79         when "111001" => MS <= "x"5" ;LS <= "x"7" ;
80         when "111010" => MS <= "x"5" ;LS <= "x"8" ;
81         when "111011" => MS <= "x"5" ;LS <= "x"9" ;
82         when OTHERS => MS <= "XXXX" ;LS <= "XXXX" ;
83
84     end case;
85
86 end process;
87
88 end BIN_TO_BD_ARC;
89
90

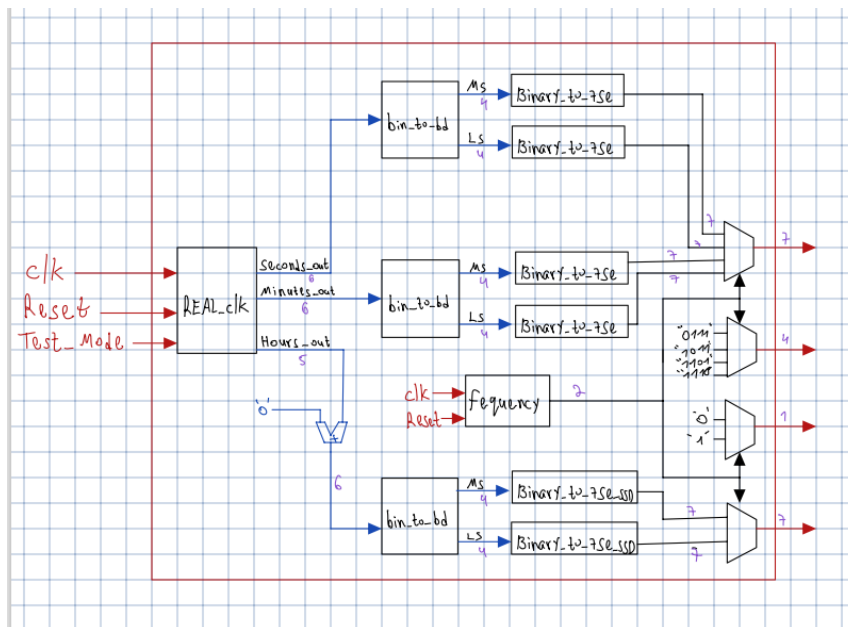
```

(ד)

אכן הקוד בלוח שונה מהקוד בSSD משום שהם הפוכים.

אחד עובד בשיטת האנודה המשותפת והשני בשיטת הקטודה המשותפת.

(4) דיאגרמת בלוקים:



I/O signal	Internal signals	Direction	Description
CLOCK		in	כניסה בעלת תדירות 100 מגה הרץ
RESET		in	ריסט אנסינכרוני פעיל באחד
TEST_MODE		in	אם (1) <= מונה עד 10 אם (0) <= מונה עד 100
BASYS_ANODE_OUT[3:0]		out	שולט על כל ספרה בלוח
BASYS_SEGMENTS_OUT[6:0]		out	מחליט איזו ספרה להפעיל
SSD_CATODE_OUT		out	מחליט איזו ספרה להפעיל
SSD_SEGMENTS_OUT[6:0]		out	שולט על כל ספרה בלוח
	Seconds_out[5:0]		מחבר בין השעון לרכיב שמחלק לשני ספרות
	Minutes_out[5:0]		
	Hours_out[4:0]		
	Hours_out_fix[4:0]		
	Ms_seconds[3:0]		מחבר בין רכיב מחלק הספרות למקודד של הלוח
	Ls_seconds[3:0]		
	Ms_minutes[3:0]		
	Ls_minutes[3:0]		
	Ms_hours[3:0]		מחבר בין רכיב מחלק הספרות למקודד של SSD
	Ls_hours[3:0]		

	Seg_ms_sec[6:0]		מחבר בין המקודד למוקס
	Seg_ls_sec[6:0]		
	Seg_ms_min[6:0]		
	Seg_ls_min[6:0]		
	Seg_ms_ho[6:0]		
	Seg_ls_ho[6:0]		
	fequency[1:0]		

XDC .6

```

1
2 #####
3 # Clocks #
4 #####
5 create_clock -name CLK -period 10 -waveform {0 5} [get_ports CLK]
6
7 #####
8 # Input Delay #
9 #####
10 set_input_delay 0 -clock CLK -add_delay [get_ports RESET TEST_MODE]
11
12 #####
13 # Output Delay #
14 #####
15 set_output_delay 0 -clock CLK -add_delay [get_ports {BASYS_SEGMENTS_OUT* SSD_SEGMENTS_OUT* BASYS_ANODE_OUT* SSD_CATHODE_OUT }]
16
17 #####
18 # Pin Locations and Voltages #
19 #####
20 set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { CLK }];
21 set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports { RESET }];
22 set_property -dict { PACKAGE_PIN T17 IOSTANDARD LVCMOS33 } [get_ports { TEST_MODE }];
23 set_property -dict { PACKAGE_PIN W4 IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[0] }];
24 set_property -dict { PACKAGE_PIN V4 IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[1] }];
25 set_property -dict { PACKAGE_PIN U4 IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[2] }];
26 set_property -dict { PACKAGE_PIN U2 IOSTANDARD LVCMOS33 } [get_ports { BASYS_ANODE_OUT[3] }];
27 set_property -dict { PACKAGE_PIN W7 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[0] }];
28 set_property -dict { PACKAGE_PIN W6 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[1] }];
29 set_property -dict { PACKAGE_PIN U8 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[2] }];
30 set_property -dict { PACKAGE_PIN V8 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[3] }];
31 set_property -dict { PACKAGE_PIN U5 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[4] }];
32 set_property -dict { PACKAGE_PIN V5 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[5] }];
33 set_property -dict { PACKAGE_PIN U7 IOSTANDARD LVCMOS33 } [get_ports { BASYS_SEGMENTS_OUT[6] }];
34 set_property -dict { PACKAGE_PIN A14 IOSTANDARD LVCMOS33 } [get_ports { SSD_CATHODE_OUT }];
35 set_property -dict { PACKAGE_PIN A16 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[0] }];
36 set_property -dict { PACKAGE_PIN B15 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[1] }];
37 set_property -dict { PACKAGE_PIN B16 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[2] }];
38 set_property -dict { PACKAGE_PIN A15 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[3] }];
39 set_property -dict { PACKAGE_PIN A17 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[4] }];
40 set_property -dict { PACKAGE_PIN C15 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[5] }];
41 set_property -dict { PACKAGE_PIN C16 IOSTANDARD LVCMOS33 } [get_ports { SSD_SEGMENTS_OUT[6] }];
42 #####
43 # Configuration Settings #
44 #####
45 set_property CFGBVS VCCO [current_design]
46 set_property CONFIG_VOLTAGE 3.3 [current_design]
47

```

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity Top_level is
6  port(
7      CLK          : in std_logic;
8      RESET        : in std_logic;
9      TestMode     : in std_logic;
10     BASYS_ANODE_OUT : out std_logic_VECTOR(3 downto 0);
11     BASYS_SEGMENTS_OUT : out std_logic_VECTOR(6 downto 0);
12     SSD_CATHODE_OUT  : out std_logic;
13     SSD_SEGMENTS_OUT : out std_logic_VECTOR(6 downto 0)
14 );
15 end Top_level;
16
17 architecture ARC_Top_level of Top_level is
18
19     component REAL_CLK
20     port
21     (
22         CLK          : in std_logic;
23         RESET        : in std_logic;
24         TestMode     : in std_logic;
25         Seconds_Out  : out std_logic_VECTOR(5 downto 0);
26         Minutes_Out  : out std_logic_VECTOR(5 downto 0);
27         Hours_Out    : out std_logic_VECTOR(4 downto 0)
28     );
29 end component;
30
31 component BIN_TO_BD
32 port
33 (
34     VALUE_BIN : in  STD_LOGIC_VECTOR(5 downto 0);
35     MS        : out STD_LOGIC_VECTOR(3 downto 0);
36     LS        : out STD_LOGIC_VECTOR(3 downto 0)
37 );
38 end component;
39
40 component BINARY_TO_7SEGMENT
41 port
42 (
43     B_IN      : in  STD_LOGIC_VECTOR(3 downto 0);
44     seven_segment : out STD_LOGIC_VECTOR(6 downto 0)
45 );
46 end component;
47
48 component BINARY_TO_7SEGMENT_SSD
49 port
50 (
51     B_IN      : in  STD_LOGIC_VECTOR(3 downto 0);
52     seven_segment : out STD_LOGIC_VECTOR(6 downto 0)
53 );
54 end component;
55
56 --inertial signals--
57 --REAL_CLK--
58 signal Seconds_Out : std_logic_VECTOR(5 downto 0);
59 signal Minutes_Out : std_logic_VECTOR(5 downto 0);
60 signal Hours_Out   : std_logic_VECTOR(4 downto 0);
61 signal Hours_Out_with0 : std_logic_VECTOR(5 downto 0);
62
63 --BIN_TO_BD--
64 signal MS_Seconds : STD_LOGIC_VECTOR(3 downto 0);
65 signal LS_Seconds : STD_LOGIC_VECTOR(3 downto 0);
66 signal MS_Minutes : STD_LOGIC_VECTOR(3 downto 0);
67 signal LS_Minutes : STD_LOGIC_VECTOR(3 downto 0);
68 signal MS_Hours   : STD_LOGIC_VECTOR(3 downto 0);
69 signal LS_Hours   : STD_LOGIC_VECTOR(3 downto 0);

```

```

70 --BINARY_TO_7SEGMENT--
71 signal Sec_7segment_MS : STD_LOGIC_VECTOR(6 downto 0);
72 signal Sec_7segment_LS : STD_LOGIC_VECTOR(6 downto 0);
73 signal Min_7segment_MS : STD_LOGIC_VECTOR(6 downto 0);
74 signal Min_7segment_LS : STD_LOGIC_VECTOR(6 downto 0);
75 --BINARY_TO_7SEGMENT_SSD--
76 signal Hours_7segment_MS : STD_LOGIC_VECTOR(6 downto 0);
77 signal Hours_7segment_LS : STD_LOGIC_VECTOR(6 downto 0);
78 --frequency--
79 signal frequency : STD_LOGIC_VECTOR(1 downto 0);
80
81 begin
82     Hours_Out_with0 <= '0' & Hours_Out;
83
84     L0: REAL_CLK          PORT map(CLK, RESET, TestMode, Seconds_Out, Minutes_Out, Hours_Out);
85
86     L1: BIN_TO_BD          PORT map(Seconds_Out, MS_Seconds, LS_Seconds);
87     L2: BIN_TO_BD          PORT map(Minutes_Out, MS_Minutes, LS_Minutes);
88     L3: BIN_TO_BD          PORT map(Hours_Out_with0, MS_Hours, LS_Hours );
89
90     L4: BINARY_TO_7SEGMENT PORT map(MS_Seconds, Sec_7segment_MS );
91     L5: BINARY_TO_7SEGMENT PORT map(LS_Seconds, Sec_7segment_LS );
92     L6: BINARY_TO_7SEGMENT PORT map(MS_Minutes, Min_7segment_MS );
93     L7: BINARY_TO_7SEGMENT PORT map(LS_Minutes, Min_7segment_LS );
94
95     L8: BINARY_TO_7SEGMENT_SSD PORT map(MS_Hours, Hours_7segment_MS );
96     L9: BINARY_TO_7SEGMENT_SSD PORT map(LS_Hours, Hours_7segment_LS );
97
98     process
99     begin
100         wait until rising_edge(CLK);
101         if (RESET = '1') then
102             frequency <= (others => '0');
103         else
104             frequency <= frequency + 1;
105         end if;
106     end process;
107
108     process(frequency, Sec_7segment_MS, Sec_7segment_LS, Min_7segment_MS, Min_7segment_LS)
109     begin
110         case frequency is
111             when "00" =>
112                 BASYS_ANODE_OUT <= "0111";
113                 BASYS_SEGMENTS_OUT <= Min_7segment_MS;
114             when "01" =>
115                 BASYS_ANODE_OUT <= "1011";
116                 BASYS_SEGMENTS_OUT <= Min_7segment_LS;
117             when "10" =>
118                 BASYS_ANODE_OUT <= "1101";
119                 BASYS_SEGMENTS_OUT <= Sec_7segment_MS;
120             when "11" =>
121                 BASYS_ANODE_OUT <= "1110";
122                 BASYS_SEGMENTS_OUT <= Sec_7segment_LS;
123             when others =>
124                 BASYS_ANODE_OUT <= "XXXX";
125                 BASYS_SEGMENTS_OUT <= "XXXXXXX";
126         end case;
127     end process;
128
129     process(frequency(0), Hours_7segment_MS, Hours_7segment_LS)
130     begin
131         case frequency(0) is
132             when "0" =>
133                 SSD_CATHODE_OUT <= "0";
134                 SSD_SEGMENTS_OUT <= Hours_7segment_MS;
135             when "1" =>
136                 SSD_CATHODE_OUT <= "1";
137                 SSD_SEGMENTS_OUT <= Hours_7segment_LS;
138             end case;
139         end process;
140     end process;
141 end process;

```

(ב) נמצא למעלה בשאלה 3 סעיפים ב,ג

ג) REAL CLK

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity REAL_CLK is
6  port(
7    CLK : in std_logic;
8    RESET : in std_logic;
9    TestMode : in std_logic;
10   Seconds_Out: out std_logic_VECTOR(5 downto 0);
11   Minutes_Out: out std_logic_VECTOR(5 downto 0);
12   Hours_Out: out std_logic_VECTOR(4 downto 0)
13 );
14
15 end REAL_CLK;
16
17 architecture ARC_REAL_CLK of REAL_CLK is
18   signal Q_Seconds : std_logic_VECTOR(5 downto 0);
19   signal Q_Minutes : std_logic_VECTOR(5 downto 0);
20   signal Q_Hours : std_logic_VECTOR(4 downto 0);
21   signal Q : std_logic_VECTOR(26 downto 0);
22   signal TC_TimeBase : std_logic;
23 begin
24   -----tc time base and mux for test mode
25   process
26   begin
27     wait until rising_edge(CLK);
28     if (RESET = '1') then
29       Q <= (others => '0');
30       TC_TimeBase <= '0';
31     elsif (TestMode = '1') then
32       if (Q < 9) then
33         Q <= Q + 1;
34         TC_TimeBase <= '0';
35       else
36         Q <= (others => '0');
37         TC_TimeBase <= '1';
38       end if;
39     elsif (TestMode = '0') then
40       if (Q < 99999999) then
41         Q <= Q + 1;
42         TC_TimeBase <= '0';
43       else
44         Q <= (others => '0');
45         TC_TimeBase <= '1';
46       end if;
47     end if;
48   end process;
49
50   ----- time counting up to seconds
51   The_Seconds: process
52   begin
53     wait until rising_edge(CLK);
54     if (RESET = '1') then
```

```
52   begin
53     wait until rising_edge(CLK);
54     if (RESET = '1') then
55       Q_Seconds <= (others => '0');
56     else
57       if (TC_TimeBase = '1') then
58         if (Q_Seconds < 59) then
59           Q_Seconds <= Q_Seconds + 1;
60         else
61           Q_Seconds <= (others => '0');
62         end if;
63       end if;
64     end if;
65   end process;
66   Seconds_Out <= Q_Seconds;
67
68   ----- time counting up to minutes
69   The_Minutes: process
70   begin
71     wait until rising_edge(CLK);
72     if (RESET = '1') then
73       Q_Minutes <= (others => '0');
74     else
75       if ((TC_TimeBase = '1') and (Q_Seconds = 59)) then
76         if (Q_Minutes < 59) then
77           Q_Minutes <= Q_Minutes + 1;
78         else
79           Q_Minutes <= (others => '0');
80         end if;
81       end if;
82     end if;
83   end process;
84   Minutes_Out <= Q_Minutes;
85
86   ----- time counting up to hours
87   The_Hours: process
88   begin
89     wait until rising_edge(CLK);
90     if (RESET = '1') then
91       Q_HOURS <= (others => '0');
92     else
93       if ((TC_TIMEBASE = '1') and (Q_MINUTES = 59) and (Q_SECONDS = 59)) then
94         if (Q_HOURS < 23) then
95           Q_HOURS <= Q_HOURS + 1;
96         else
97           Q_HOURS <= (others => '0');
98         end if;
99       end if;
100     end if;
101   end process;
102   Hours_Out <= Q_Hours;
103
104 end ARC_REAL_CLK;
105
```


(8

TESTBENCH:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity TB_Top_level is
6  end TB_Top_level;
7
8  architecture TB_ARC_Top_level of TB_Top_level is
9  component Top_level
10 port(
11     CLK           : in std_logic;
12     RESET         : in std_logic;
13     TestMode      : in std_logic;
14     BASYS_ANODE_OUT : out std_logic_VECTOR(3 downto 0);
15     BASYS_SEGMENTS_OUT : out std_logic_VECTOR(6 downto 0);
16     SSD_CATHODE_OUT : out std_logic;
17     SSD_SEGMENTS_OUT : out std_logic_VECTOR(6 downto 0)
18 );
19 end component;
20
21 signal TB_CLK           : std_logic = '0' ;
22 signal TB_RESET         : std_logic;
23 signal TB_TestMode      : std_logic;
24 signal TB_BASYS_ANODE_OUT : std_logic_VECTOR(3 downto 0);
25 signal TB_BASYS_SEGMENTS_OUT : std_logic_VECTOR(6 downto 0);
26 signal TB_SSD_CATHODE_OUT : std_logic;
27 signal TB_SSD_SEGMENTS_OUT : std_logic_VECTOR(6 downto 0);
28
29 begin
30     DUT : Top_level
31 PORT map(
32     CLK           => TB_CLK,
33     RESET         => TB_RESET,
34     TestMode      => TB_TestMode,
35     BASYS_ANODE_OUT => TB_BASYS_ANODE_OUT,
36     BASYS_SEGMENTS_OUT => TB_BASYS_SEGMENTS_OUT,
37     SSD_CATHODE_OUT => TB_SSD_CATHODE_OUT,
38     SSD_SEGMENTS_OUT => TB_SSD_SEGMENTS_OUT
39 );
40     TB_RESET <= '1',
41             '0' after 13 ns;
42     TB_TestMode <= '1';
43
44 process
45 begin
46     wait for 5 ns;
47     TB_CLK <= not(TB_CLK);
48 end process;
49 -- בשביל שהקלט יפלט כל שניה
50 end TB_ARC_Top_level;
```